

Large Language Models – Deep Interview Guide

1. What is a Large Language Model (LLM)?

A Large Language Model (LLM) is a neural network trained to model the probability distribution of sequences of tokens. At its mathematical core, an LLM learns $P(\text{token}_t \mid \text{token}_1 \dots \text{token}_{t-1})$. Rather than storing facts like a database, it learns statistical regularities across vast corpora of text. During inference, it generates outputs by sampling from probabilities of possible next tokens. This predictive framing explains both its impressive fluency and its limitations.

LLMs are typically trained using self-supervised learning. Massive datasets are constructed where parts of text are hidden, and the model learns to predict missing tokens. No human labels are required for most of the training. Over time, the model develops internal representations that encode grammar, semantics, world knowledge patterns, and reasoning-like behaviors — emergent properties arising from scale rather than explicit programming.

Importantly, an LLM does not possess symbolic understanding or grounded knowledge. It manipulates vector representations optimized for likelihood, not truth. When the model produces a confident but incorrect answer, this is not a bug in reasoning but a consequence of probabilistic generation. Interviews often probe this distinction: models optimize linguistic plausibility, not epistemic certainty.

Modern LLMs are built on transformer architectures, enabling parallel computation and long-range dependency modeling. Scale (parameters, data, compute) plays a crucial role. As models grow, capabilities emerge non-linearly — better reasoning, translation, coding, summarization. Understanding this scaling behavior is valuable in interviews discussing model performance trends.

2. Tokenization

Tokenization converts raw text into discrete units that the model can process. Contrary to intuition, tokens are not always words. Most systems use subword tokenization (BPE, WordPiece, SentencePiece), breaking text into fragments. This allows efficient handling of rare words, misspellings, and multilingual text while keeping vocabulary size manageable.

Because pricing, context limits, and truncation are token-based, tokenization has real engineering consequences. A seemingly short sentence may expand into many tokens. Special characters, whitespace, emojis, and code can drastically change token counts. Interviews often test whether candidates understand why prompt cost and memory limits are governed by tokens rather than characters.

Tokenization also explains surprising model behaviors. The model's perception of similarity depends on token patterns, not human word boundaries. Prompt engineering techniques sometimes exploit token structure — for example, avoiding awkward splits or leveraging repeated token prefixes to steer outputs.

Different models use different tokenizers, meaning identical text may produce different token counts across providers. This impacts latency, pricing, and truncation. Practical interview discussions may involve estimating token usage, designing token budgets, or debugging truncation failures.

3. Embeddings

Embeddings map tokens (or sentences/documents) into continuous vector spaces. These vectors capture semantic relationships: similar meanings cluster together geometrically. Embeddings power retrieval systems, semantic search, clustering, and recommendation engines.

Within LLMs, embeddings are learned representations optimized during training. They serve as the input layer, transforming symbolic tokens into numeric form. During RAG pipelines, separate embedding models often encode queries and documents for similarity search in vector databases.

Embedding arithmetic illustrates encoded structure. Classic examples like $\text{king} - \text{man} + \text{woman} \approx \text{queen}$ demonstrate how semantic directions emerge. Interviews may ask candidates to explain cosine similarity, vector distance metrics, or how embeddings enable retrieval without keyword matching.

Good system design requires understanding embedding drift, domain mismatch, and chunk granularity. Poor chunking can degrade retrieval accuracy. Candidates are often evaluated on their ability to reason about embedding quality and retrieval performance.

4. Transformer Architecture

Transformers revolutionized NLP by replacing recurrence with attention. Instead of processing tokens sequentially, transformers evaluate all tokens in parallel, dramatically improving training efficiency and capturing long-range dependencies.

The architecture consists of stacked layers containing multi-head self-attention and feed-forward networks. Residual connections and normalization stabilize deep training. Each layer refines contextual representations of tokens.

Parallelization allows scaling to billions of parameters. This is why transformers dominate LLM development. Interviews frequently include questions comparing transformers vs RNNs/LSTMs, emphasizing gradient flow, memory limitations, and computational efficiency.

Understanding transformers is essential for discussing inference optimizations (KV cache), complexity ($O(n^2)$), and architectural innovations (sparse attention, Mixture-of-Experts).

5. Attention Mechanism

Attention computes how strongly tokens relate to each other. Queries, Keys, and Values interact via dot products, producing attention scores. Softmax normalizes these scores into weights, determining information flow.

Self-attention allows every token to reference every other token. This enables contextual understanding, coreference resolution, and dependency modeling. Multi-head attention learns diverse relational patterns simultaneously.

Attention complexity grows quadratically with sequence length, impacting latency and memory. Interviewers often ask candidates to explain why long prompts are expensive and how techniques like sparse attention mitigate costs.

Conceptual clarity on attention is a high-priority interview requirement. Candidates should confidently describe Q/K/V, scaling factors, masking, and interpretability debates.

6. Positional Encoding

Because transformers lack inherent sequence order, positional encoding injects token position information. Without it, word order would be lost.

Techniques include sinusoidal encodings and learned positional embeddings. These allow the model to differentiate 'dog bites man' from 'man bites dog'.

Advanced interviews may probe relative positional encodings or RoPE (Rotary Position Embeddings), commonly used in modern LLMs for better extrapolation.

Position handling becomes critical for long-context models and memory extensions.

7. Context Window

The context window defines how many tokens the model can consider simultaneously. It represents a hard memory boundary during inference.

When inputs exceed the window, earlier tokens are truncated. This leads to forgotten instructions or degraded coherence.

Engineering solutions include chunking, summarization, sliding windows, and retrieval augmentation.

Interview discussions often involve designing systems that manage long documents without exceeding token limits.

8. Hallucinations

Hallucinations occur when the model generates plausible but incorrect outputs. This arises because LLMs predict likelihood, not truth.

They are more likely under ambiguity, missing knowledge, or high temperature settings.

Mitigation strategies include RAG, verification loops, tool use, constrained decoding, and guardrails.

Interviewers frequently ask candidates to diagnose and reduce hallucinations.

Interview Q&A; Highlights

Why transformers outperform RNNs? Transformers enable parallel computation, better gradient flow, and long-range dependency modeling without recurrence bottlenecks.

Why hallucinations happen? Because the model predicts statistically likely tokens rather than verifying factual accuracy.

How reduce hallucinations? Use RAG, external tools, verification loops, constrained decoding, and careful prompt design.

What is context window? The maximum token length the model can attend to during inference.